

Documentação TMP117 V2

Francisco Almeida

March 2026

Conteúdo

1	Introduction	2
2	Ligações à placa	2
3	Posição dos sensores	4
4	Codigos	5
4.1	Codigo Arduino	5
4.2	Codigo Matlab	13

1 Introduction

Este documento tem como objetivo apresentar o código desenvolvido para para os sensores TMP117, assim como gráficos de vários testes efetuados com este mesmo código. Vai ser também apresentado a posição dos sensores assim como as ligações que cada sensor efetua para com a ESP-32 sendo os endereços dos sensores os definidos no documento apresentado anteriormente.

2 Ligações à placa

As ligações Sensor-Placa foram as seguintes:

ESP32	Sensor 1	Sensor 2	Sensor 3
	0x48	0x4A	0x4B
3V3	Vin	Vin	Vin
GND	GND	GND	GND
P21	SCA	SCA	SCA
P22	SCL	SCL	SCL
P27	INT	INT	INT

Tabela 1: Ligações Esp32→Sensores.

Estas ligações foram feitas com a ajuda de uma placa branca para poder fazer as ligações dos sensores à ESP-32 como mostra na seguinte imagem:

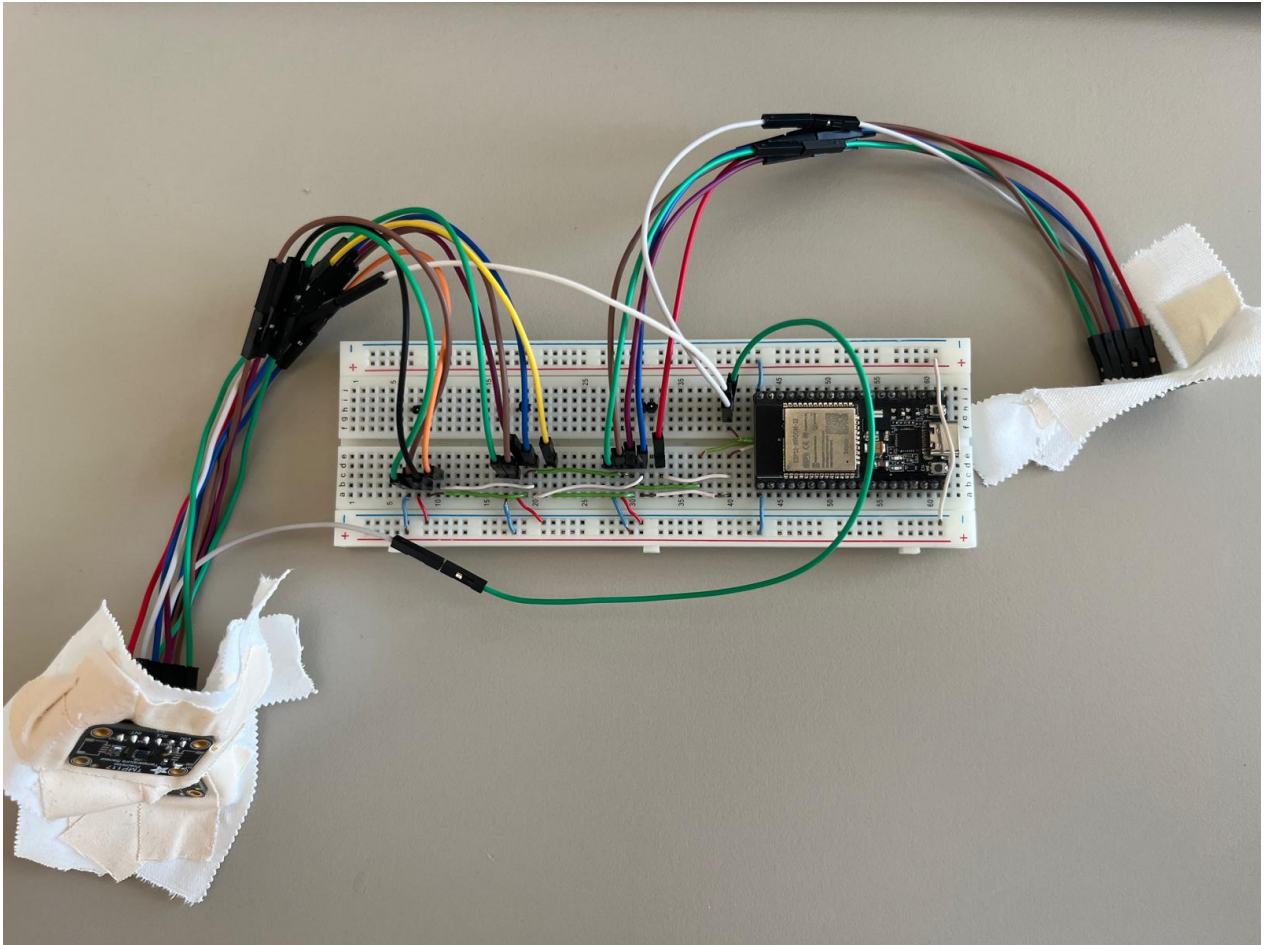


Figura 1: Ligação dos sensores à placa branca.

3 Posição dos sensores

A posição dos sensores escolhida teve como base o documento anterior a este, concluindo que a melhor posição para os sensores seria na zona do peito perto dos mamilos. Dito isto, colocamos dois sensores a cerca de um dedo do mamilo esquerdo, sendo que 1 sensor esta na parte horizontal inferior e outro esta na parte direito vertical. O terceiro sensor esta localizado no peito direito à mesma distancia do mamilo que os outros sendo que esta na posição horizontal inferior do mamilo.

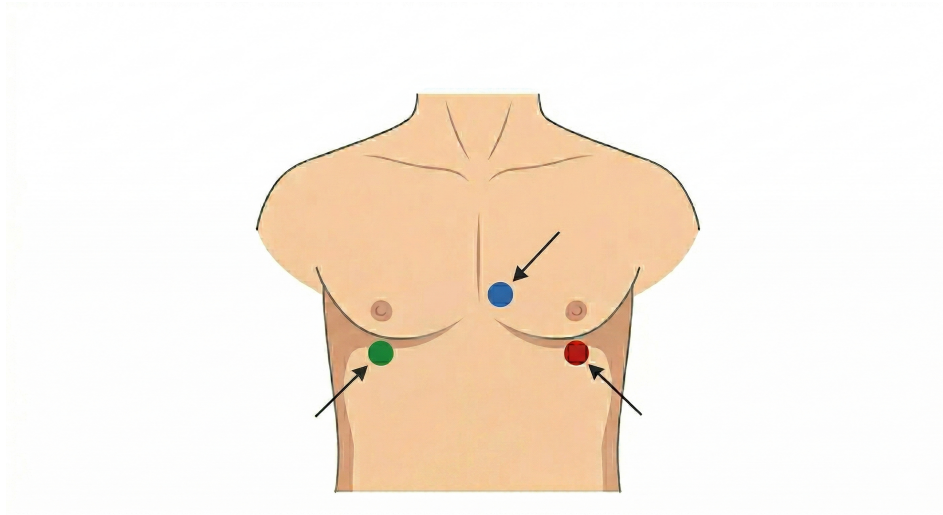


Figura 2: Posição dos sensores no corpo.

4 Codigos

4.1 Codigo Arduino

Este codigo implementa o algoritmo apresentado na imagem seguinte:

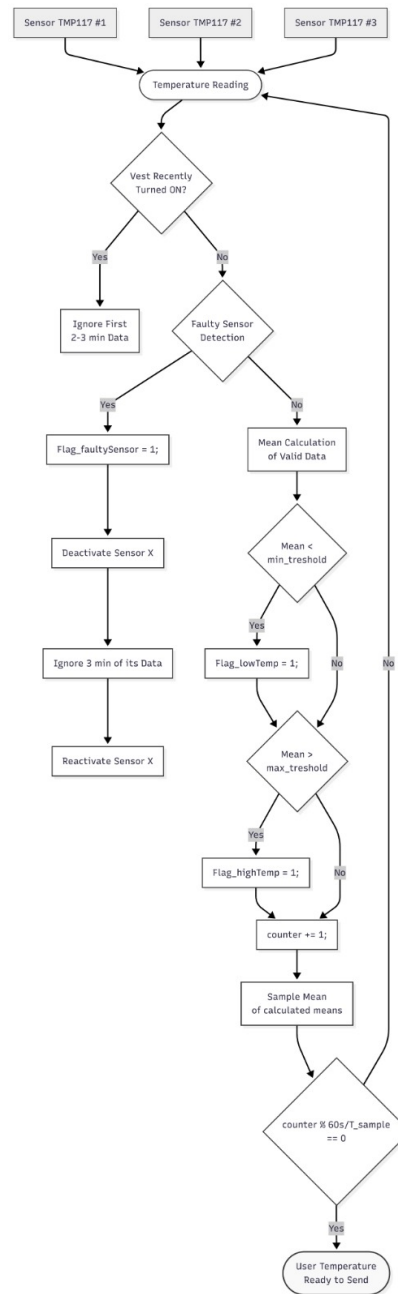


Figura 3: Algoritmo referente à parte dos sensores de temperatura.

O algoritmo apresenta algumas condições pensadas pela equipa para tentar receber a melhor informação possível.

Este algoritmo apresenta a condição de se o colete foi ligado recentemente e se a condição for verdadeira ignora a informação recebida durante três minutos, a escolha deste tempo foi efetuada após varios testes onde os sensores estabilizavam a temperatura após três minutos de os encostarmos ao corpo.

Após esta condição, verificamos se os sensores tem valores relativamente próximos usando o "Falty Sensor Detector" esta condição ativa uma flag cada vez que um sensor estiver com medições muito diferentes dos outros sensores e desativa o sensor durante três minutos, mais uma vez o tempo do sensor estabilizar a temperatura e passado esse tempo o sensor volta a ativar. Esta condição faz a diferença entre as dois sensores de cada vez e analisando se a diferença é superior a um valor predefinido, no caso entre os sensores que estão próximos (0x48 e 0x4A) o limite é 1°C e destes dois para o terceiro sensor (0x4B) é de 2,5°C.

Após esta análise de qualidade dos sensores e assumindo que estão todos ativos e com valores validos, calculamos a media das temperaturas e de seguida comparamos com valores minimos e maximos que nos alertam para hipotermia ou hipertermia sendo estes intervalos 34°C e 38°C respetivamente.

Por fim fazemos uma media amostral com as amostras medias que obtivemos durante um minuto, após este calculo ser realizado enviamos tanto as temperaturas de cada sensor, a media dos sensores a cada dez segundos (Tempo de amostragem) e a media amostral das medias calculadas.

```
#include <Wire.h>
#include <Adafruit_TMP117.h>
#include <Adafruit_Sensor.h>
#include <math.h>

#define I2C_SDA 21
#define I2C_SCL 22
#define ALERT_PIN 27

#define PERIOD_MS 10000UL
#define IGNORE_TIME_MS 180000UL // 3 minutos

#define MIN_THRESHOLD 34.0f
#define MAX_THRESHOLD 38.0f

#define CLOSE_PAIR_THRESHOLD 1.0f // limite entre 0x48 e 0x4B
#define FAR_SENSOR_THRESHOLD 2.5f // limite do 0x4A para os outros

#define TMP117_ADDR_1 0x48
#define TMP117_ADDR_2 0x4A
#define TMP117_ADDR_3 0x4B

Adafruit_TMP117 tmp1;
Adafruit_TMP117 tmp2;
Adafruit_TMP117 tmp3;

struct SensorNode {
  Adafruit_TMP117* dev;
  uint8_t addr;
  bool ok;
  float lastTemp;
};

SensorNode sensors[3] = {
  { &tmp1, TMP117_ADDR_1, false, NAN }, // index 0 -> 0x48
  { &tmp2, TMP117_ADDR_2, false, NAN }, // index 1 -> 0x4A
  { &tmp3, TMP117_ADDR_3, false, NAN } // index 2 -> 0x4B
```

```

};

volatile bool isrFlag = false;

unsigned long lastKick = 0;
unsigned long systemStartMs = 0;    // instante em que o sistema arrancou
bool waitingForData = false;

bool Flag_lowTemp = false;
bool Flag_highTemp = false;
bool Flag_faultySensor = false;

int faultySensorIndex = -1;

bool sensorIgnored[3] = { false, false, false };
unsigned long ignoreUntil[3] = { 0, 0, 0 };

uint32_t counter = 0;
float sumCalculatedMeans = 0.0f;
uint32_t numCalculatedMeans = 0;

float userTemperatureToSend = NAN;
bool userTemperatureReadyToSend = false;

void IRAM_ATTR onAlert() {
    isrFlag = true;
}

static bool readReg16(uint8_t a, uint8_t reg, uint16_t &v) {
    Wire.beginTransaction(a);
    Wire.write(reg);
    if (Wire.endTransmission(false) != 0) return false;
    if (Wire.requestFrom((int)a, 2) != 2) return false;
    v = ((uint16_t)Wire.read() << 8) | Wire.read();
    return true;
}

static bool writeReg16(uint8_t a, uint8_t reg, uint16_t v) {
    Wire.beginTransaction(a);
    Wire.write(reg);
    Wire.write((uint8_t)(v >> 8));
    Wire.write((uint8_t)(v & 0xFF));
    return Wire.endTransmission() == 0;
}

bool initTMP117(SensorNode &s) {
    s.ok = s.dev->begin(s.addr, &Wire);
    Serial.printf("INIT | sensor_addr=0x%02X | status=%s\r\n", s.addr, s.ok ? "
    OK" : "FAIL");
    if (!s.ok) return false;

    s.dev->interruptsActiveLow(true);

    uint16_t cfg = 0;
    if (readReg16(s.addr, 0x01, cfg)) {
        cfg |= 0x0040;
        if (!writeReg16(s.addr, 0x01, cfg)) {
            Serial.printf("ERROR | type=WriteCFG | sensor_addr=0x%02X\r\n", s.addr);
            s.ok = false;
            return false;
        }
    }
    Serial.printf("CFG | sensor_addr=0x%02X | config=0x%04X\r\n", s.addr, cfg)

```

```

    ;
} else {
    Serial.printf("ERROR | type=ReadCFG | sensor_addr=0x%02X\r\n", s.addr);
    s.ok = false;
    return false;
}

s.dev->setMeasurementMode(TMP117_MODE_SHUTDOWN);
return true;
}

bool inStartupIgnoreWindow(unsigned long now) {
    return (now - systemStartMs) < IGNORE_TIME_MS;
}

void kickAllSensors() {
    isrFlag = false;
    waitingForData = true;
    lastKick = millis();

    for (int i = 0; i < 3; i++) {
        if (sensors[i].ok) {
            sensors[i].dev->setMeasurementMode(TMP117_MODE_ONE_SHOT);
        }
    }

    Serial.printf("KICK | ms=%lu\r\n", lastKick);
}

void updateIgnoredSensors(unsigned long now) {
    for (int i = 0; i < 3; i++) {
        if (sensorIgnored[i] && now >= ignoreUntil[i]) {
            sensorIgnored[i] = false;
            ignoreUntil[i] = 0;
            Serial.printf("SENSOR_REACTIVATED | sensor_addr=0x%02X | ms=%lu\r\n",
                sensors[i].addr, now);
        }
    }
}

void ignoreSensor(int idx, unsigned long now) {
    sensorIgnored[idx] = true;
    ignoreUntil[idx] = now + IGNORE_TIME_MS;
    Flag_faultySensor = true;
    faultySensorIndex = idx;

    Serial.printf("SENSOR_IGNORED | sensor_addr=0x%02X | ms=%lu | ignoreUntil=%lu\r\n",
        sensors[idx].addr, now, ignoreUntil[idx]);
}

bool faultySensorDetection(bool validMask[3], float temps[3], unsigned long
    now) {
    Flag_faultySensor = false;
    faultySensorIndex = -1;

    // S corre a dete o quando os 3 sensores est o dispon veis e n o
    ignorados
    if (!validMask[0] || !validMask[1] || !validMask[2]) {
        Serial.println("FAULT_CHECK | status=SKIPPED | reason=
            not_all_sensors_available");
        return false;
    }
}

```



```

}

// 0 -> 0x48
// 1 -> 0x4A
// 2 -> 0x4B
float d_48_4A = fabs(temps[0] - temps[1]);
float d_48_4B = fabs(temps[0] - temps[2]);
float d_4A_4B = fabs(temps[1] - temps[2]);

Serial.printf("FAULT_CHECK | d_48_4A=%.3f | d_48_4B=%.3f | d_4A_4B=%.3f\r\n",
    "
    ",
    d_48_4A, d_48_4B, d_4A_4B);

// Regra 1: se 0x48 e 0x4B diferirem mais de 1 C ,
// ignora aquele cuja diferen a para 0x4A for maior
if (d_48_4B > CLOSE_PAIR_THRESHOLD) {
    if (d_48_4A > d_4A_4B) {
        validMask[0] = false;
        ignoreSensor(0, now);
        Serial.println("FAULT_DETECTED | faultySensor=0x48");
        return true;
    } else if (d_4A_4B > d_48_4A) {
        validMask[2] = false;
        ignoreSensor(2, now);
        Serial.println("FAULT_DETECTED | faultySensor=0x4B");
        return true;
    } else {
        Serial.println("FAULT_DETECTED | faultySensor=NONE | reason=
        tie_against_0x4A");
        return false;
    }
}

// Regra 2: se 0x4A estiver demasiado longe dos outros dois, ignora 0x4A
if (d_48_4A > FAR_SENSOR_THRESHOLD && d_4A_4B > FAR_SENSOR_THRESHOLD) {
    validMask[1] = false;
    ignoreSensor(1, now);
    Serial.println("FAULT_DETECTED | faultySensor=0x4A");
    return true;
}

Serial.println("FAULT_DETECTED | faultySensor=NONE");
return false;
}

bool readAllSensors(float &mean, uint8_t &validCount, unsigned long now) {
    float temps[3] = { NAN, NAN, NAN };
    bool validMask[3] = { false, false, false };

    updateIgnoredSensors(now);

    for (int i = 0; i < 3; i++) {
        if (!sensors[i].ok) {
            sensors[i].lastTemp = NAN;
            continue;
        }

        sensors_event_t ev;
        sensors[i].dev->getEvent(&ev);

        if (!isnan(ev.temperature)) {
            sensors[i].lastTemp = ev.temperature;

```

```

        // S entra para calculo se n o estiver ignorado
        if (!sensorIgnored[i]) {
            temps[i] = ev.temperature;
            validMask[i] = true;
        }
        else {
            sensors[i].lastTemp = NAN;
        }
    }

    // Tentar detetar sensor defeituoso apenas quando os 3 est o ativos
    faultySensorDetection(validMask, temps, now);

    float sum = 0.0f;
    validCount = 0;

    for (int i = 0; i < 3; i++) {
        if (validMask[i]) {
            sum += temps[i];
            validCount++;
        }
    }

    if (validCount == 0) {
        mean = NAN;
        return false;
    }

    mean = sum / validCount;
    return true;
}

void printSensorTemps(unsigned long now) {
    Serial.printf("RAW | ms=%lu", now);

    for (int i = 0; i < 3; i++) {
        Serial.printf(" | sensor_%d_addr=0x%02X", i + 1, sensors[i].addr);

        if (sensors[i].ok && !isnan(sensors[i].lastTemp)) {
            Serial.printf(" | sensor_%d_temp=%.3f", i + 1, sensors[i].lastTemp);
        } else {
            Serial.printf(" | sensor_%d_temp=NA", i + 1);
        }

        Serial.printf(" | sensor_%d_ignored=%d", i + 1, sensorIgnored[i] ? 1 : 0);

        if (sensorIgnored[i]) {
            Serial.printf(" | sensor_%d_ignoreUntil=%lu", i + 1, ignoreUntil[i]);
        }
    }

    Serial.printf(" | Flag_faultySensor=%d", Flag_faultySensor ? 1 : 0);

    if (faultySensorIndex >= 0) {
        Serial.printf(" | faultySensor_addr=0x%02X", sensors[faultySensorIndex].
            addr);
    } else {
        Serial.print(" | faultySensor_addr=NONE");
    }

    Serial.println();
}

```

```

}

void processAlgorithm(float mean, uint8_t validCount, unsigned long now) {
    Flag_lowTemp = (mean < MIN_THRESHOLD);
    Flag_highTemp = (mean > MAX_THRESHOLD);

    counter++;

    sumCalculatedMeans += mean;
    numCalculatedMeans++;

    float sampleMean = sumCalculatedMeans / numCalculatedMeans;

    userTemperatureReadyToSend = false;

    const uint32_t samplesPerMinute = 60000UL / PERIOD_MS;

    if (samplesPerMinute > 0 && (counter % samplesPerMinute == 0)) {
        userTemperatureToSend = sampleMean;
        userTemperatureReadyToSend = true;

        Serial.printf("USER_TEMP_READY | ms=%lu | userTemperatureToSend=%.3f\r\n",
            now, userTemperatureToSend);

        sumCalculatedMeans = 0.0f;
        numCalculatedMeans = 0;
    }

    Serial.printf("DATA | ms=%lu | mean=%.3f | validCount=%u | Flag_lowTemp=%d |
        Flag_highTemp=%d | Flag_faultySensor=%d | counter=%lu | sampleMean=%.3f
        | readyToSend=%d\r\n",
        now,
        mean,
        validCount,
        Flag_lowTemp ? 1 : 0,
        Flag_highTemp ? 1 : 0,
        Flag_faultySensor ? 1 : 0,
        counter,
        sampleMean,
        userTemperatureReadyToSend ? 1 : 0);
}

void setup() {
    Serial.begin(115200);
    delay(1500);
    Serial.println("START");

    systemStartMs = millis();    // marca o arranque do sistema

    Wire.begin(I2C_SDA, I2C_SCL);
    delay(100);

    initTMP117(sensors[0]);
    initTMP117(sensors[1]);
    initTMP117(sensors[2]);

    int activeSensors = 0;
    for (int i = 0; i < 3; i++) {
        if (sensors[i].ok) activeSensors++;
    }

    if (activeSensors == 0) {

```

```

        Serial.println("ERROR | type=NoSensorFound");
        return;
    }

    pinMode(ALERT_PIN, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(ALERT_PIN), onAlert, FALLING);

    Serial.println("HEADER | ms | mean | validCount | Flag_lowTemp |
        Flag_highTemp | Flag_faultySensor | counter | sampleMean | readyToSend")
        ;

    kickAllSensors();
}

void loop() {
    unsigned long now = millis();

    if (!waitingForData && (now - lastKick >= PERIOD_MS)) {
        kickAllSensors();
    }

    if (waitingForData && (isrFlag || digitalRead(ALERT_PIN) == LOW)) {
        isrFlag = false;
        waitingForData = false;

        float mean = NAN;
        uint8_t validCount = 0;
        unsigned long tRead = millis();

        if (readAllSensors(mean, validCount, tRead)) {
            printSensorTemps(tRead);

            // Ignorar os primeiros 3 minutos ap s arranque
            if (inStartupIgnoreWindow(tRead)) {
                unsigned long remaining = IGNORE_TIME_MS - (tRead - systemStartMs);

                Serial.printf("STARTUP_IGNORE | ms=%lu | mean=%.3f | validCount=%u |
                    remainingMs=%lu\r\n",
                        tRead, mean, validCount, remaining);

                userTemperatureReadyToSend = false;
                return;
            }

            processAlgorithm(mean, validCount, tRead);
        } else {
            Serial.printf("DATA | ms=%lu | mean=NA | validCount=0 | Flag_lowTemp=%d
                | Flag_highTemp=%d | Flag_faultySensor=%d | counter=%lu | sampleMean
                =NA | readyToSend=0\r\n",
                    tRead,
                    Flag_lowTemp ? 1 : 0,
                    Flag_highTemp ? 1 : 0,
                    Flag_faultySensor ? 1 : 0,
                    counter);
        }
    }
}

```

4.2 Codigo Matlab

Este codigo possibilita a criação e visualização dos graficos com os valores anteriormente retirados

```
clear; clc; close all;

%% CONFIGURACAO SERIAL
porta = "COM4";          % <-- ALTERA para a tua porta
baudrate = 115200;

s = serialport(porta, baudrate);
configureTerminator(s, "LF");
flush(s);

%% CONSTANTES
STARTUP_IGNORE_MS = 180000;    % 3 minutos

%% VARIAVEIS
tempo_s = [];
sensor1 = [];
sensor2 = [];
sensor3 = [];
meanSample = [];

minuteTime = [];
minuteMean = [];

current_ms = NaN;
current_s1 = NaN;
current_s2 = NaN;
current_s3 = NaN;

current_s1_ignored = 0;
current_s2_ignored = 0;
current_s3_ignored = 0;

current_faulty_addr = "NONE";

startupFinished = false;

%% FIGURA
f = figure('Name','TMP117 Monitor','NumberTitle','off');

ax1 = subplot(2,1,1);
hold(ax1, 'on');
grid(ax1, 'on');
title(ax1, 'Temperaturas dos Sensores e Media por Amostra');
xlabel(ax1, 'Tempo (s)');
ylabel(ax1, 'Temperatura (^{\circ}C)');

h1 = plot(ax1, NaN, NaN, '-o', 'DisplayName', 'Sensor 0x48');
h2 = plot(ax1, NaN, NaN, '-o', 'DisplayName', 'Sensor 0x4A');
h3 = plot(ax1, NaN, NaN, '-o', 'DisplayName', 'Sensor 0x4B');
h4 = plot(ax1, NaN, NaN, '-', ...
    'Color', [1 0.4 0.7], ...
    'LineWidth', 2, ...
    'DisplayName', 'Media por Amostra');

legend(ax1, 'Location', 'best');

ax2 = subplot(2,1,2);
hold(ax2, 'on');
```

```

grid(ax2, 'on');
title(ax2, 'Media Amostral (1 minuto)');
xlabel(ax2, 'Tempo (s)');
ylabel(ax2, 'Temperatura ( $\circ$ C)');

h5 = plot(ax2, NaN, NaN, '-sr', 'LineWidth', 2, 'DisplayName', 'Media Amostral
');
legend(ax2, 'Location', 'best');

% texto de estado
statusBox = annotation('textbox', [0.13 0.92 0.75 0.05], ...
    'String', 'Estado: a aguardar fim da janela inicial de 3 minutos', ...
    'FitBoxToText', 'on', ...
    'EdgeColor', 'white');

disp("A ler dados da serial... Prime Ctrl+C para parar.");

%% CICLO PRINCIPAL
while ishandle(f)
    try
        line = strtrim(readline(s));
        disp(line);

        % =====
        % LINHA RAW
        % =====
        if startsWith(line, "RAW")
            msTok = regexp(line, 'ms=(\d+)', 'tokens', 'once');

            s1Tok = regexp(line, 'sensor_1_temp=([-\d\.NA]+)', 'tokens', 'once
');
            s2Tok = regexp(line, 'sensor_2_temp=([-\d\.NA]+)', 'tokens', 'once
');
            s3Tok = regexp(line, 'sensor_3_temp=([-\d\.NA]+)', 'tokens', 'once
');

            s1IgnTok = regexp(line, 'sensor_1_ignored=(\d+)', 'tokens', 'once
');
            s2IgnTok = regexp(line, 'sensor_2_ignored=(\d+)', 'tokens', 'once
');
            s3IgnTok = regexp(line, 'sensor_3_ignored=(\d+)', 'tokens', 'once
');

            faultyTok = regexp(line, 'faultySensor_addr=([A-Za-z0-9x]+)', '
tokens', 'once');

            if ~isempty(msTok)
                current_ms = str2double(msTok{1}) / 1000;
            end

            if ~isempty(s1Tok), current_s1 = parseValue(s1Tok{1}); end
            if ~isempty(s2Tok), current_s2 = parseValue(s2Tok{1}); end
            if ~isempty(s3Tok), current_s3 = parseValue(s3Tok{1}); end

            if ~isempty(s1IgnTok), current_s1_ignored = str2double(s1IgnTok
{1}); end
            if ~isempty(s2IgnTok), current_s2_ignored = str2double(s2IgnTok
{1}); end
            if ~isempty(s3IgnTok), current_s3_ignored = str2double(s3IgnTok
{1}); end

            if ~isempty(faultyTok)

```

```

        current_faulty_addr = string(faultyTok{1});
    else
        current_faulty_addr = "NONE";
    end
end

% =====
% LINHA STARTUP_IGNORE
% Ignora completamente os primeiros 3 minutos
% =====
if startsWith(line, "STARTUP_IGNORE")
    msTok = regexp(line, 'ms=(\d+)', 'tokens', 'once');
    remTok = regexp(line, 'remainingMs=(\d+)', 'tokens', 'once');

    if ~isempty(msTok)
        current_ms = str2double(msTok{1}) / 1000;
    end

    if ~isempty(remTok)
        remaining_s = str2double(remTok{1}) / 1000;
        statusText = sprintf(['Estado: janela inicial ativa | ' ...
                               'dados ignorados | tempo restante = %.1f'
                               's'], remaining_s);
    else
        statusText = 'Estado: janela inicial ativa | dados ignorados';
    end

    set(statusBox, 'String', statusText);
    drawnow limitrate;
    continue;
end

% =====
% LINHA DATA
% Aqui recalculamos a media apenas com sensores nao ignorados
% =====
if startsWith(line, "DATA")
    msTok = regexp(line, 'ms=(\d+)', 'tokens', 'once');

    if ~isempty(msTok)
        t = str2double(msTok{1}) / 1000;
    else
        t = current_ms;
    end

    % Protecao adicional: nao aceitar nada nos primeiros 3 minutos
    if ~isnan(t) && t < (STARTUP_IGNORE_MS / 1000)
        set(statusBox, 'String', ...
            'Estado: dados recebidos mas ainda dentro da janela'
            ' inicial de 3 minutos');
        drawnow limitrate;
        continue;
    end

    startupFinished = true;

    % aplicar ignorados -> NaN no grafico
    s1_plot = current_s1;
    s2_plot = current_s2;
    s3_plot = current_s3;

    if current_s1_ignored == 1, s1_plot = NaN; end
end

```

```

if current_s2_ignored == 1, s2_plot = NaN; end
if current_s3_ignored == 1, s3_plot = NaN; end

vals = [s1_plot, s2_plot, s3_plot];
m = mean(vals, 'omitnan');

% Se todos estiverem NaN, nao acrescenta ponto invalido
if all(isnan(vals))
    set(statusBox, 'String', ...
        'Estado: todos os sensores ativos para o grafico estao
        indisponiveis/ignorados');
    drawnow limitrate;
    continue;
end

tempo_s(end+1) = t;
sensor1(end+1) = s1_plot;
sensor2(end+1) = s2_plot;
sensor3(end+1) = s3_plot;
meanSample(end+1) = m;

% atualizar grafico superior
set(h1, 'XData', tempo_s, 'YData', sensor1);
set(h2, 'XData', tempo_s, 'YData', sensor2);
set(h3, 'XData', tempo_s, 'YData', sensor3);
set(h4, 'XData', tempo_s, 'YData', meanSample);

% texto de estado
ignoredList = {};
if current_s1_ignored == 1, ignoredList{end+1} = '0x48'; end
if current_s2_ignored == 1, ignoredList{end+1} = '0x4A'; end
if current_s3_ignored == 1, ignoredList{end+1} = '0x4B'; end

if isempty(ignoredList)
    statusText = "Estado: aquisi o normal | nenhum sensor
    ignorado";
else
    statusText = "Estado: aquisi o normal | sensor(es) ignorado
    (s): " + ...
        strjoin(ignoredList, ', ') + ...
        " | faultySensor_addr=" + current_faulty_addr;
end

set(statusBox, 'String', char(statusText));

drawnow limitrate;
end

% =====
% LINHA USER_TEMP_READY
% =====
if startsWith(line, "USER_TEMP_READY")
    msTok = regexp(line, 'ms=(\d+)', 'tokens', 'once');
    uTok = regexp(line, 'userTemperatureToSend=(-\d\.NA)+', 'tokens', 'once');

    if ~isempty(msTok)
        tMin = str2double(msTok{1}) / 1000;
    else
        tMin = NaN;
    end
end

```



```

        % Nao aceitar medias de 1 minuto durante a janela inicial
        if ~startupFinished || (~isnan(tMin) && tMin < (STARTUP_IGNORE_MS
            / 1000))
            continue;
        end

        if ~isempty(uTok)
            mMin = parseValue(uTok{1});
        else
            mMin = NaN;
        end

        if ~isnan(tMin) && ~isnan(mMin)
            minuteTime(end+1) = tMin;
            minuteMean(end+1) = mMin;

            set(h5, 'XData', minuteTime, 'YData', minuteMean);
            drawnow limitrate;
        end
    end

    catch ME
        warning("Erro na leitura/processamento: %s", ME.message);
    end
end

%% FUNCAO AUXILIAR
function v = parseValue(str)
    if strcmpi(str, 'NA')
        v = NaN;
    else
        v = str2double(str);
    end
end
end

```